

App Development Project Report

1. Student Details

Name: Lokagagan K S

Roll Number: 24f3004113

Email: 24f3004113@ds.study.iitm.ac.in

About Me: I am Lokagagan KS, a student of BS in Data Science and Application at IIT Madras interested in learning new things. Through this Project I aim to gain knowledge of Web Development. Through this project shall make myself aware of the Flask, HTML, Java script, Vue.js, Celery, Redis, Caching, Mailhog and Bootstrap.

2. Project Details

Project Title: Placement Portal Application - V2

Problem Statement:

To design a web application that an Admin to manage the Placement Activity, where Company can register and manage Job Drives, and Student can register and Apply available Job Drives.

Approach:

Using Flask for backend api functionality with defined function at different endpoints, which is directed to the Vue.js frontend with index.html and Bootstrap for Styling. With Roles as Admin, Company and Create implementing their Functionalities as Create Drives, Edit and Update Drives and Approve company to create drive and manage them and completing the shortlist, schedule interview for applied Students can apply for all Approved and Open Job Drives and manage Profile, Admin as superuser to manage and Monitor all Activities.

3. AI/LLM Declaration

I used **ChatGPT** to assist in file upload and its handling, Search and filtering features of Companies, Students and drives. To Learn and understand Celery part, few Vue.js method The extent of AI/LLM usage is below 20%, limited to few code suggestions and few debugging.

All final implementation logic, debugging, and integration were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Vue.js	Frontend UI
Bootstrap 5	Frontend styling and responsive design
SQLite	Local database
Redis	Caching (with Flask-Caching) and message broker (With Celery Worker)
Celery	Mail processing and Schedule Jobs
Flask Sessions / JWT	Authentication
MailHog	Local email testing

5. Database Schema / ER Diagram

Tables:

1. **User**
 - uid, email, password, role
2. **Student**
 - sid, user_id, name, phone, course, cgpa, graduation_year, skills, resume, blacklisted
3. **Company**
 - cid, user_id, company_name, industry, location, hr_contact, website, approval_status, blacklisted
4. **Drive**
 - did, company_id, job_title, description, course, min_cgpa, graduation_year, salary, application_deadline, approval_status, status
5. **Application**
 - aid, student_id, drive_id, application_date, status, feedback
6. **ApplicationHistory**
 - hid, application_id, status, feedback, updated_at

7. Placement

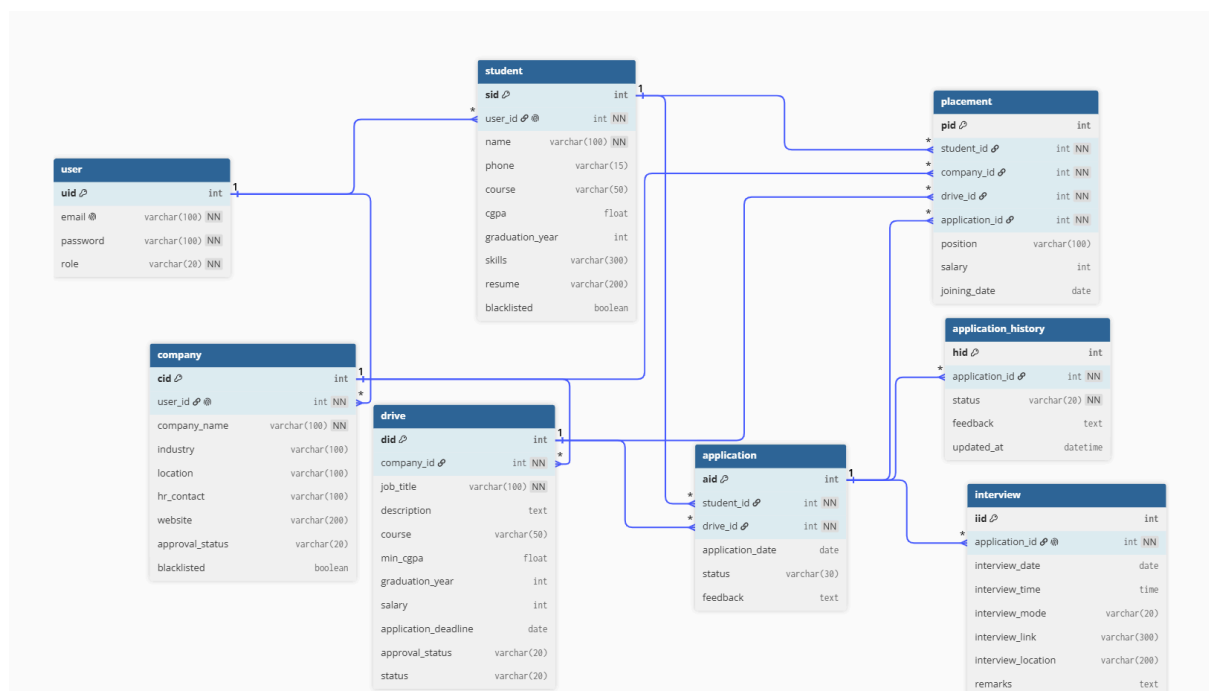
- pid, student_id, company_id, drive_id, application_id, position, salary, joining_date

8. Interview

- iid, application_id, interview_date, interview_time, interview_mode, interview_link, interview_location, remarks

Relationships:

- One-to-One → User → Student
- One-to-One → User → Company
- One-to-Many → Company → Drive
- One-to-Many → Student → Application
- One-to-Many → Drive → Application
- One-to-Many → Application → ApplicationHistory
- One-to-One → Application → Interview
- One-to-One → Application → Placement
- One-to-Many → Student → Placement
- One-to-Many → Company → Placement
- One-to-Many → Drive → Placement



6. API Resource Endpoints

Endpoint	Method	Description
/login	POST	User authentication
/student/dashboard	GET	Student dashboard
/student/applications	GET	View applications
/student/application/<aid>/history	GET	View application history
/student/history	GET	View placement history
/company/application/<aid>	GET	View application
/company/application/<aid>/update	PUT	Update application status
/company/application/<aid>/interview	POST	Add interview details
/admin/application/<aid>/history	GET	View application history

7. Architecture and Features

Architecture Overview:

- **app.py** – main Flask application entry point
- **db.py** – database models using Flask-SQLAlchemy
- **routes/login.py** – Flask Blueprint for user login
- **routes/reg.py** – Flask Blueprint for user registration
- **routes/admin.py** – Flask Blueprint for admin dashboard and management functions
- **routes/student.py** – Flask Blueprint for student dashboard, applications and placement history
- **routes/company.py** – Flask Blueprint for company dashboard, placement drives and application management
- **celery_worker.py** – Celery worker configuration
- **task.py** - background task processing
- **/frontend** – Vue.js frontend application
- **/frontend/src/views** – Vue pages for Admin, Student and Company dashboards

- **SQLite database** – stores users, students, companies, placement drives, applications, application history, interviews and placements
- **Flask Session/JWT** – used for authentication and authorization
- **Redis and Flask Caching** – used for caching and as the Celery message broker/backend

Implemented Features:

Separate login for **Admin, Companies, and Students**, with registration available for Companies and Students.

Admin can **approve/reject companies**, manage students, companies, placement drives, and applications.

Companies can manage their **profile, create and update placement drives, and view applicants**.

Students can **search and view placement drives, apply for eligible drives, and track application status**.

Companies can update application status such as **Shortlisted, Interview, Selected, or Rejected** and provide feedback.

Interview details can be scheduled and updated by companies, with email notifications sent to students.

The system maintains a **complete application status history** with status, feedback, date, and time.

Selected students are recorded as **placements** with position, salary, and joining date.

Students can manage their **profile and resume**, and view applications, interviews, placements, and application history.

The system prevents unauthorized access using **JWT authentication and Flask sessions**.

Celery and Redis are used for background and scheduled tasks such as interview reminders and monthly placement reports.

Email notifications are sent for important events such as application updates, interview updates, and approved placement drives.

Redis caching with Flask-Caching is used to improve performance for frequently accessed data.

Basic **responsive Bootstrap-based UI** is provided for Admin, Company, and Student interfaces.

8. Video Presentation

Drive Link:  mad2 24f3004113 project demo.mp4

<https://drive.google.com/file/d/1kyKyuoj33HW4TILHpuAenZq-kHoLEQWA/view?usp=sharing>